B.TECH CAPSTONE DEFENSE · ACADEMIC YEAR 2026–2027

TrustChain-AI

A Decentralized Trust Intelligence Framework for Secure Autonomous AI Agent Collaboration

Department of Computer Engineering · Mukesh Patel School of Technology Management & Engineering (MPSTME)
SVKM's NMIMS (Deemed-to-be University), Mumbai

01 · IDENTITY LAYER

W3C did:key & Ed25519

Decentralized cryptographic public-key identity generation without central CA dependency. W3C Verifiable Credentials with LD-Proofs.

02 · TRUST INTELLIGENCE

Continuous Temporal EMA

Multi-factor sensor telemetry (Success, Compliance, Latency). Mathematical Exponential Moving Average with 4 discrete states.

03 · ZERO-TRUST CONTROL

Adaptive Policy Gateway

Operation-level privilege isolation (Privileged vs Standard). Transparent multi-factor agent matching and React 19 SOC portal.

PROJECT PRESENTERS & MENTORSHIP

Submitted by: [STUDENT NAME 1] (Roll No: [ROLL NUMBER 1]) & [STUDENT NAME 2] (Roll No: [ROLL NUMBER 2])

Project Mentor: [MENTOR NAME] | B.Tech Final-Year Capstone Project | Session 2026–2027

The Point-in-Time Security Failure in Multi-Agent Systems

Three Critical Failure Modes of Static Authentication

1. The Binary Point-of-Entry Fallacy

Traditional access control treats authentication as a one-time gate: an agent presents a valid credential or API key at t=0 and receives persistent operational access. However, validity at connection time provides zero guarantee of runtime execution safety.

2. Post-Authentication Behavioral Drift

Autonomous agents are prone to model hallucination, API timeout degradation, configuration tampering, and prompt injection subversion. Once past the initial authentication barrier, a degraded agent continues executing privileged workloads undetected.

3. Cascading Compromise in Multi-Agent Workflows

In sequential and collaborative pipelines (Agent A -> Agent B -> Agent C), output from an unmonitored degraded agent is consumed as trusted input by subsequent nodes, compounding errors and undermining systemic integrity.

Security Paradigm Shift: Static vs. Continuous Zero-Trust

TRADITIONAL PARADIGM: BLIND RUNTIME TRUST

```
[Static API Key / Token]  ──>  [Access Granted (t=0)]  ──>
[Unchecked Execution]  ──>  [Silent Failure / Breach]
```

- Binary access: Agent is either 100% trusted or completely blocked.
- No runtime behavioral monitoring or telemetry feedback loop.
- Privileges persist even during catastrophic performance degradation.

TRUSTCHAIN-AI PARADIGM: CONTINUOUS ZERO-TRUST

```
[W3C did:key] ──> [Sensor Telemetry] ──> [EMA Trust Scoring] ──>
[Adaptive Gating] ──> [Audit Trail]
```

- Multi-factor evaluation: Success rate (S), compliance (C), latency (L).
- Temporal smoothing prevents single-event volatility: $T(t) = 0.7 \cdot T(t-1) + 0.3 \cdot P(t)$.
- Automatic dynamic isolation: Quarantines degraded agents ($T < 0.50$).

Formal Engineering Objectives & Scope Governance

Six Core Technical Objectives (Implemented & Verified)

Obj 1 · Cryptographic Identity Genesis
Implement W3C did:key decentralized identifiers using Ed25519 elliptic-curve keypairs with multicodec encoding (0xED01).

Obj 2 · Verifiable Credential Operations
Formulate W3C JSON-LD Verifiable Credentials with URDNA2015 canonicalization and Ed25519Signature2020 proof verification.

Obj 3 · Multi-Factor Telemetry Ingestion
Capture runtime sensor events evaluating task success rate S(t), policy compliance C(t), and SLA latency penalty L(t).

Obj 4 · Temporal EMA Trust Engine
Develop mathematical Exponential Moving Average smoothing $T(t) = 0.7 \cdot T(t-1) + 0.3 \cdot P(t)$ across 4 discrete states.

Obj 5 · Adaptive Policy Enforcement
Dynamically gate PRIVILEGED operations (workflow execution, delegation) vs STANDARD operations across trust states.

Obj 6 · Trust-Aware Discovery & Matching
Implement capability discovery and task-aware matching algorithm: $\text{MatchScore} = 0.4T + 0.3(r/5) + 0.2\sigma + 0.1(1-\lambda)$.

Implementation Scope vs. Explicit Non-Claims

IN-SCOPE IMPLEMENTED CAPABILITIES

- FastAPI backend with 10 routers, Pydantic v2 validation, and CORS.
- Local SQLite Verifiable Data Registry (WAL mode) with 13 tables.
- React 19 SOC operations dashboard with dark/light theme support.
- 80 passing automated unit/integration tests executed via pytest.
- Deterministic LocalAgentConnector for structured capability execution.

EXPLICIT NON-CLAIMS & BOUNDARIES

- No Distributed Blockchain: Local relational SQLite VDR; distributed ledger anchoring is identified as future scope.
- No Live Commercial LLMs: Default test harness uses deterministic synthetic responses; live LLM APIs are future work.
- Sequential Workflows Verified: The database schema models parallel nodes, but execution tests strictly validate sequential pipelines.

Literature Survey & Research Gap Analysis

THEMATIC LITERATURE SYNTHESIS (15 VERIFIED CITATIONS)

Surveying peer-reviewed literature across Self-Sovereign Identity (W3C, RFC 8037), Distributed Trust (EigenTrust, Jøsang), Zero Trust Architecture (NIST SP 800-207), and Multi-Agent Systems (FIPA).

1. SSI & DECENTRALIZED IDENTITY

W3C DID Core (2022) [1]
W3C VC Model (2022) [2]
RFC 8037 Ed25519 (2017) [8]

STRENGTH:
Cryptographic decentralization; user-controlled verifiable claims.

LIMITATION:
Human-centric; static credentials lack runtime behavioral coupling.

TRUSTCHAIN-AI CONTRIBUTION:
Couples did:key with continuous behavioral telemetry scoring.

2. DISTRIBUTED TRUST SYSTEMS

Jøsang et al. (2007) [3]
EigenTrust / Kamvar (2003) [4]
Mui et al. (2002) [5]

STRENGTH:
Reputation aggregation in distributed P2P topologies.

LIMITATION:
Assumes static peer relationships; lacks capability & SLA matching.

TRUSTCHAIN-AI CONTRIBUTION:
Integrates success, compliance, and SLA latency into EMA trust.

3. ZERO TRUST ARCHITECTURE

NIST SP 800-207 (2020) [15]
Zhang et al. (2020) [11]
Bernabe et al. (2019) [10]

STRENGTH:
Continuous verification; elimination of implicit network trust.

LIMITATION:
Focused on network microsegmentation, not agent reasoning.

TRUSTCHAIN-AI CONTRIBUTION:
Operationalizes ZTA for autonomous agent collaboration pipelines.

4. MULTI-AGENT COORDINATION

FIPA Agent Mgmt (2004) [12]
Shoker (2017) [13]
Nitti et al. (2014) [6]

STRENGTH:
Agent Directory Facilitator (DF) capability registry.

LIMITATION:
No runtime behavioral gating, quarantine, or revocation.

TRUSTCHAIN-AI CONTRIBUTION:
Adaptive gating separating Privileged vs Standard operations.

TrustChain-AI Architectural Paradigm & System Contributions

The Seven Integrated Architectural Pillars

- **1. Identity Genesis:**
  Ed25519 keypair generation multicodec prefixed (0xED01) to form W3C did:key.
- **2. Verifiable Credentials:**
  JSON-LD credential issuance with URDNA2015 canonicalization & LD-proofs.
- **3. Runtime Telemetry Ingestion:**
  Event sensors capturing success, compliance breaches, and latency in milliseconds.
- **4. Trust Intelligence Core:**
  Mathematical sliding-window scoring Pi(t) coupled with temporal EMA smoothing Ti(t).
- **5. Adaptive Policy Gateway:**
  Operation-level privilege isolation across TRUSTED, SUSPICIOUS, QUARANTINED, REVOKED.
- **6. Trust-Aware Discovery & Matching:**
  Multi-factor candidate ranking: MatchScore = 0.4T + 0.3(r/5) + 0.2σ + 0.1(1-λ).
- **7. Quad-Dimensional Audit Trail:**
  Tamper-evident logs across user sessions, state changes, telemetry, and collaborations.

Three Principal Engineering Contributions

CONTRIBUTION 1: SSI + DYNAMIC TRUST SYNTHESIS

Proves that decentralized cryptographic identity can be successfully coupled with dynamic behavioral telemetry to govern autonomous AI agents without relying on centralized identity providers.

CONTRIBUTION 2: ADAPTIVE ZERO-TRUST POLICY GATING

Establishes a dual-layer access model: human role-based access control (ADMIN, OPERATOR, VIEWER) combined with autonomous agent runtime trust states that dynamically quarantine degraded agents.

CONTRIBUTION 3: EXPLAINABLE MULTI-FACTOR MATCHING

Formulates a transparent, verifiable candidate ranking equation balancing trust, historical reputation, task success, and latency, backed by a responsive React 19 SOC operations interface.

Five-Tier System Architecture & Operational Topology

TIER 1 · PRESENTATION TIER (PORT 5173) | React 19 · Vite · Tailwind CSS · Recharts · Lucide Icons

- SOC Security Dashboard: Fleet trust distribution, live health probes, telemetry volume visualizers.
- Agent Explorer & Investigation Drawer: Real-time risk posture visualizers, deep audit transition inspections.
- Capability Discovery Center: Multi-filter capability search, trust threshold slider, diagnostic eligibility rationales.
- Task-Driven Matching Center: Workload prompt input, Top Match spotlight card, transparent score breakdown.

TIER 2 · API GATEWAY TIER (PORT 8050) | FastAPI · Uvicorn ASGI · Pydantic v2 · JWT (HS256) · Argon2id

- Security Headers Middleware: X-Request-ID tracing, nosniff, DENY clickjacking, strict referrer policies.
- Dual Authentication Gate: Human session JWT tokens (Argon2id hashing) & Agent Bearer token verification.
- RESTful Router Modules: /auth, /vdr (credentials), /trust (telemetry & state), /registry (discovery & matching).

TIER 3 · TRUST INTELLIGENCE TIER | TrustEngine · AdaptivePolicyEnforcer · LocalAgentConnector

- Trust Engine Core: Sliding-window evaluation (S, C, L), instantaneous score P(t), temporal EMA smoothing T(t).
- Adaptive Policy Gateway: Operation-level authorization gating (Privileged vs Standard) across 4 trust states.
- Local Agent Connector: Deterministic structured response engine & outbound HTTP POST collaboration bridge.

TIER 4 · IDENTITY & CREDENTIALS | W3C did:key · PyNaCl Ed25519 · PyLD URDNA2015 · Base58

- DID Generator: Ed25519 32-byte key generation, 0xED01 multicodec prefixing, base58 multibase encoding.
- Verifiable Credentials: W3C JSON-LD credential issuance, URDNA2015 dataset normalization, Ed25519 proofs.

TIER 5 · PERSISTENCE & VDR TIER | SQLite 3 (WAL Mode) · SQLAlchemy 2.0 ORM · Relational VDR

- Relational Verifiable Data Registry: 13 normalized tables storing credentials, trust states, telemetry, and audits.
- ACID Guarantee & Concurrency: Write-Ahead Logging (WAL) mode supporting concurrent readers and sub-5ms probes.

Cryptographic Agent Identity & Verifiable Credentials Lifecycle

W3C did:key Decentralized Identifier Architecture

1. Algorithmic Identity Construction

• Public Key Cryptography: Generates raw 32-byte Ed25519 keypair using libsodium via PyNaCl library.
• Multicodec Prefix: Prepends 2-byte varint 0xED01 indicating Ed25519 public key.
• Base58 Encoding: Encodes prefixed bytes with Bitcoin base58 alphabet, prefixed with 'z'.
• Formal Format: did:key:z6MkpTHR8VNsBxYAAWHut2Geadd9jSwuBV8xRo...

2. Self-Sovereign Decentralization

• No Central Authority: Identifiers are generated algorithmically without requiring DNS registration, central certificate authorities, or proprietary registration fees.
• Deterministic Resolution: Any node in the network resolves public key directly from the DID string itself without querying an external registry.

3. Identity vs. Trust Boundary

• Critical Principle: Possession of a valid did:key proves cryptographic ownership of the private key. It does NOT prove current behavioral trustworthiness.

W3C Verifiable Credentials & LD-Proof Lifecycle

W3C JSON-LD CREDENTIAL STRUCTURE

```
{
  "@context": ["https://www.w3.org/2018/credentials/v1"],
  "id": "urn:uuid:7f3b4c12-...",
  "type": ["VerifiableCredential", "AgentCapabilityCredential"],
  "issuer": "did:key:z6Mku..." (Authority DID),
  "credentialSubject": { "id": "did:key:...", "capabilities": ["analysis"] },
  "proof": { "type": "Ed25519Signature2020", "proofValue": "..." }
}
```

CANONICALIZATION & CRYPTOGRAPHIC PROOF

• URDNA2015 Normalization: Uses PyLD to transform JSON-LD into deterministic N-Quads dataset prior to hashing, resolving JSON key reordering issues.
• Ed25519Signature2020: Hashes normalized string via SHA-256 and signs with issuer private key; verifies against issuer's resolved public key.

CREDENTIAL LIFECYCLE PIPELINE

[Keypair Gen] ──> [DID Creation] ──> [VC Issuance] ──> [VDR Storage] ──> [Verification]

Runtime Sensor Telemetry & Instantaneous Behavioral Scoring

Runtime Telemetry Sensor Ingestion Model

STANDARDIZED TELEMETRY EVENT SCHEMA

```
TelemetryEvent(
  agent_did: str,          # Target agent identifier
  task_id: str,            # Unique execution UUID
  success: bool,           # Boolean task completion flag
  compliance_event: str,   # Policy breach descriptor (None if OK)
  latency_ms: int,         # Measured execution duration
  timestamp: datetime      # UTC observation timestamp
)
```

THREE EVALUATED BEHAVIORAL METRICS (SLIDING WINDOW W=10)

1. Task Success Rate $S_i(t) \in [0.0, 1.0]$:
   Ratio of successful executions over the sliding observation window.
2. Protocol Compliance Rate $C_i(t) \in [0.0, 1.0]$:
   Ratio of events with zero compliance breaches or protocol violations.
3. Normalized Latency Penalty $L_i(t) \in [0.0, 1.0]$:
   Normalized response duration: min(avg_latency / 2000ms, 1.0).

Instantaneous Performance Score Formulation P_i(t)

THE INSTANTANEOUS BEHAVIORAL EQUATION

$$P_i(t) = 0.6 \cdot S_i(t) + 0.3 \cdot C_i(t) + 0.1 \cdot (1.0 - L_i(t))$$

Constraints: $w1 + w2 + w3 = 1.0$ | $w1, w2, w3 \in [0, 1]$ | $P_i(t) \in [0.0, 1.0]$

PARAMETER WEIGHTING RATIONALE

- Task Success Weight (w1 = 0.60):
  Functional correctness is the paramount requirement. If an agent fails to complete reasoning orimplementation tasks, trust must degrade substantially.
- Protocol Compliance Weight (w2 = 0.30):
  Enforces safety boundaries, API protocol conformance, and non-repudiation. Compliance violations trigger acute trust reductions.
- Latency Penalty Weight (w3 = 0.10):
  Encourages operational efficiency. Benchmarked against a 2000ms SLA threshold; responses exceeding 2000ms receive zero latency bonus.

Temporal EMA Smoothing & Behavioral State Transition Machine

Temporal Exponential Moving Average (EMA)

TEMPORAL UPDATE FORMULATION

$$T_i(t) = \lambda \cdot T_i(t-1) + (1.0 - \lambda) \cdot P_i(t)$$

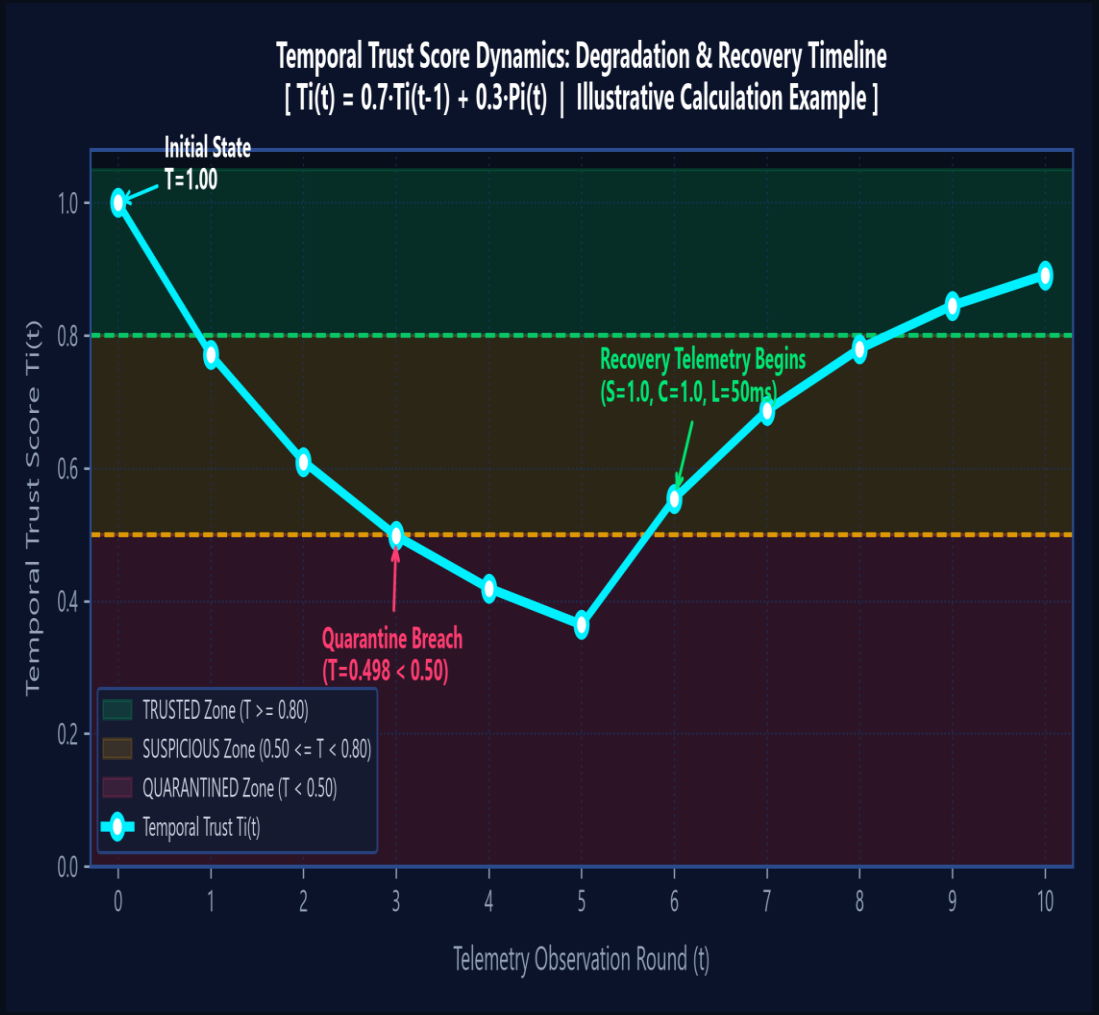
With verified default $\lambda = 0.70$:

$$T_i(t) = 0.7 \cdot T_i(t-1) + 0.3 \cdot P_i(t)$$

- Initial Baseline: $T_i(0) = 1.00$ (Optimistic initial trust).
- Historical Memory Factor ($\lambda = 0.70$): Prevents single anomalous errors from destabilizing operational networks while ensuring sustained degradation triggers isolation.

FOUR DISCRETE BEHAVIORAL STATES

- TRUSTED ($T >= 0.80$): Full authorization for standard & privileged tasks.
- SUSPICIOUS ($0.50 <= T < 0.80$): Warning state; privileged actions blocked.
- QUARANTINED ($T < 0.50$): Automated isolation; all actions denied (HTTP 403).
- REVOKED (Terminal): Administrative kill-switch; cannot recover via telemetry.



Dual-Layer Defense: Human RBAC & Agent Trust-State Gating

Dual-Layer Governance: Human RBAC vs. Agent Trust

LAYER 1: HUMAN ROLE-BASED ACCESS CONTROL (RBAC)

- ADMIN: Full platform governance, user management, policy updates, terminal agent revocation.
- OPERATOR: Operational execution, initiating workflows, requesting agent matches, monitoring logs.
- VIEWER: Read-only inspection of fleet posture, telemetry charts, and security audit trails.

LAYER 2: AUTONOMOUS AGENT RUNTIME TRUST GATING

- Independent of Credentials: An agent with a valid cryptographic credential is still blocked if its behavioral trust score has degraded.
- Continuous Decision Gate: Access decisions are re-evaluated dynamically on every single API invocation.

SESSION DEFENSE & CREDENTIAL INTEGRITY

- Password Security: Argon2id hashing algorithm with salt generation.
- Lockout Defense: 5 consecutive failed attempts locks account for 15 minutes.
- Token Rotation: Refresh token rotation with cryptographic hash reuse detection.

Operation Privilege Gating Matrix

OPERATION CLASSIFICATION TAXONOMY

- PRIVILEGED OPERATIONS:
  `workflow:execute, workflow:orchestrate, agent:delegate, credential:issue, admin:revoke, admin:manage, sensitive:execute`
- STANDARD OPERATIONS:
  `read, query, map:generate, task:execute, collaboration:step`

DYNAMIC DECISION MATRIX ACROSS TRUST STATES

STATE	STANDARD OP	PRIVILEGED OP	GATEWAY DECISION
TRUSTED	ALLOWED	ALLOWED	HTTP 200 (Permit)
SUSPICIOUS	ALLOWED (Audited)	DENIED (Policy)	HTTP 403 (Restricted)
QUARANTINED	DENIED (403)	DENIED (403)	HTTP 403 (Quarantine)
REVOKED	DENIED (403)	DENIED (403)	HTTP 403 (Terminal)

Zero-Trust Guarantee: A compromised key cannot execute privileged actions once behavioral degradation trips the Suspicious or Quarantined boundaries.

Agent Capability Discovery & Pre-Execution Eligibility Filter

Discovery Engine Architecture & Filters

DOMAIN ROLE: CAPABILITY & AVAILABILITY SEARCH

The Discovery Engine provides broad inventory search and pre-execution filtering. It answers: 'Which candidate agents exist in the registry that possess the required capability, are currently online, and meet baseline trust criteria?'

VERIFIED DISCOVERY CONTROLS & FILTERS

1. Capability Selector:
   Filters by verified tags (analysis, research, coding, planning, verification, compute).
2. Minimum Trust Threshold Slider:
   Dynamic real-time slider (0.00 to 1.00 in steps of 0.05).
3. Operational Availability Filter:
   AVAILABLE, BUSY, OFFLINE status filtering.
4. Endpoint Health Probe Filter:
   HEALTHY, UNHEALTHY, OFFLINE status verification.
5. Search Query Input:
   Instant filtering by DID prefix or agent display name.

Diagnostic Eligibility & Exclusion Rationale Engine

EXPLAINABLE ELIGIBILITY CLASSIFICATION

Unlike opaque registries that silently hide agents, TrustChain-AI explicitly evaluates every candidate against policy and surfaces clear diagnostic rationales.

ELIGIBLE CANDIDATE STATUS PILL

- Status: [ELIGIBLE]
- Rationale: 'Eligible: Meets capability, trust threshold, operational status & health criteria.'

EXCLUSION DIAGNOSTIC EXAMPLES

- Policy Exclusion:
  'Excluded: Under behavioral quarantine (T = 0.42 < 0.50).'
- Threshold Exclusion:
  'Excluded: Trust score (0.65) is below requested minimum (0.80).'
- Availability Exclusion:
  'Excluded: Operational status is BUSY on active collaboration.'
- Health Exclusion:
  'Excluded: Endpoint health probe returned UNHEALTHY.'

Action Shortcuts: Direct one-click 'Investigate' drawer and 'Engage in Collaboration' buttons.

Task-Driven Matchmaker & Explainable Multi-Factor Ranking

Multi-Factor Matching Algorithm Formulation

THE COMPOSITE MATCH SCORE EQUATION

$$\text{MatchScore} = 0.4 \cdot T + 0.3 \cdot (r/5.0) + 0.2 \cdot \sigma + 0.1 \cdot (1 - \lambda)$$

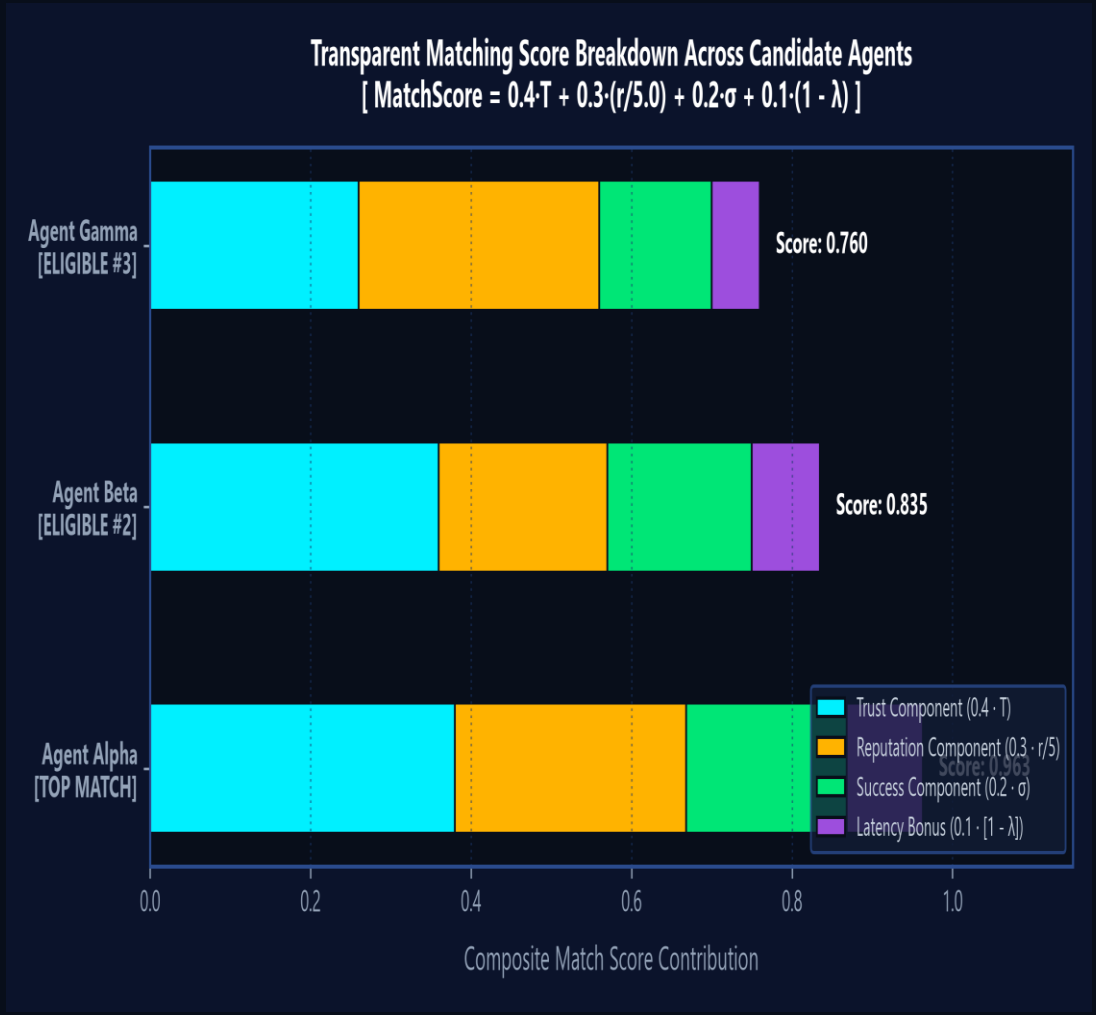
- T: Normalized temporal trust score [0.0, 1.0].
- r: Historical peer reputation rating [1.0, 5.0] normalized by /5.0.
- σ: Historical task success rate [0.0, 1.0] (successes / total_tasks).
- λ: Normalized latency penalty min(latency_ms / 2000, 1.0).

ZERO-TRUST HARD BOUNDARY ENFORCEMENT

- Immediate Policy Disqualification: If an agent is in QUARANTINED or REVOKED state, MatchScore is forced to 0.0 with 'REJECTED BY POLICY', regardless of high historical reputation.
- Privileged Restriction: If state is SUSPICIOUS and task is privileged, candidate is disqualified.

TOP MATCH RECOMMENDATION SPOTLIGHT

Highlights the highest-scoring candidate with a glowing cyan banner, mathematical breakdown chips, recommendation rationale, and one-click 'Assign Task' button.



Multi-Agent Collaboration Sessions & Sequential Pipeline Orchestration

Bilateral Collaboration Protocol & Lifecycle Management

1. Session Lifecycle State Machine

Collaboration sessions progress through deterministic states:
[PENDING] → [ACTIVE] → [COMPLETED] / [TERMINATED] / [FAILED]

- PENDING: Session initiated, mutual identity and VC exchange verified.
- ACTIVE: Bilateral task payload execution and sensor telemetry logging.
- COMPLETED: Task successfully delivered, mutual ratings recorded.
- TERMINATED: Forced termination upon trust decay or policy violation.

2. Session Entity Data Model (akta/vdr/models.py)

- session_id: UUIDv4 unique session handle.
- initiator_did & target_did: Cryptographic did:key identifiers.
- session_token: Cryptographically signed ephemeral JWT.
- collaboration_events: Granular append-only event log capturing every request, intermediate step, response payload, and sensor signature.

3. Mid-Session Trust Boundary Enforcement

If either agent's trust drops below 0.50 during a multi-turn collaboration, the session is instantly aborted, preventing tainted output propagation.

Sequential Workflow Pipeline & Deterministic Simulation Engine

SEQUENTIAL PIPELINE EXECUTION MODEL

Complex enterprise tasks are decomposed into multi-stage pipelines:
[Node 1: Ingestion] → [Node 2: Analysis] → [Node 3: Verification]

- Ordered Node Traversal: Output of Node i serves as verified input to Node i+1.
- Adaptive Authorization at Every Step: Before each node executes, the gateway verifies the agent's real-time trust state (T ≥ 0.80 for privileged steps).
- Atomic Failure Handling: If any node fails or is quarantined, the pipeline halts cleanly with full audit trail logging.

LOCAL AGENT CONNECTOR (REPRODUCIBILITY)

- Deterministic Simulation: Implemented in akta/connectors/local.py.
- Eliminates Live LLM Dependencies: Avoids flaky external API costs, network jitter, and token limits during automated test runs.
- Realistic Workload Emulation: Simulates parameterized response payloads, synthetic compute delays, and injection error codes.

PARALLEL EXECUTION BOUNDARY (ACADEMIC HONESTY)

While DAG dependencies are supported in the database schema (workflow_nodes.depends_on), current execution is strictly sequential.

Local Relational VDR Architecture & Quad-Dimensional Auditability

13-Table Relational Schema & WAL High-Throughput Storage

1. Verifiable Data Registry (VDR) Implementation

TrustChain-AI implements a local relational VDR using SQLite configured with Write-Ahead Logging (WAL mode):

• Sub-5ms Read/Write: High-throughput local operations without gas fees, block confirmation delays, or external RPC network failures.

• ACID Compliance: Atomic transaction guarantees for all trust state updates and credential issuance operations.

2. Complete 13-Table Relational Architecture

• Identity & Credentials: stored_vcs, agents

• Trust & Telemetry: agent_trust_states, telemetry_logs, trust_audit_trails

• Collaboration: collaboration_sessions, collaboration_events, agent_ratings

• Workflows: workflows, workflow_nodes

• Security & Access: users, refresh_tokens, security_audit_events

3. Academic Clarification: Relational VDR vs Blockchain

Per W3C DID specifications, a Verifiable Data Registry is a logical interface. Our implementation uses SQLite WAL for high performance. Deploying to an Ethereum/Hedera ledger represents future distributed work.

Quad-Dimensional Auditability & Verifiable Ledger Traceability

FOUR IMMUTABLE AUDIT STREAMS

TrustChain-AI maintains 4 distinct, tamper-evident audit streams providing 360-degree observability:

[DIMENSION 1] stored_vcs (Credential Lifecycle)

Records every issued, validated, or revoked W3C Verifiable Credential with cryptographic issuer signature and LD-Proof payload.

[DIMENSION 2] trust_audit_trails (State Transitions)

Immutable log of all trust updates: old_score, new_score, old_state, new_state, trigger_reason, and millisecond timestamp.

[DIMENSION 3] telemetry_logs (Runtime Interaction Events)

Granular per-task sensor telemetry: success, policy compliance, SLA latency, input hashes, and sensor signatures.

[DIMENSION 4] security_audit_events (Security Gateway)

Authentication attempts, account lockouts, RBAC privilege violations, and adaptive policy rejection intercepts.

Operational SOC Portal Demonstration & Interface Verification

Enterprise SOC Dashboard & Navigation Architecture

1. Modern Frontend Architecture

• React 19 + Vite: Ultra-responsive component architecture compiled to production in 26.2s via Vite.
• Dark Navy SOC Theme: Styled with Tailwind CSS (#080E1A background, #111C35 panels, #00F0FF accents) with seamless light mode toggle.
• Lucide Icons & Headless UI: Accessible, keyboard-navigable operational controls.

2. Five Dedicated Operational Modules

• Overview / Metrics Dashboard: High-level KPI summary, trust distribution pie/bar charts, active collaboration count, live security alert feeds.
• Agent Explorer (/agents): Grid view of all registered agents with real-time status badges (TRUSTED, SUSPICIOUS, QUARANTINED, REVOKED).
• Discovery Center (/discovery): Capability tag selector, dynamic trust threshold slider (0.0–1.0), and diagnostic eligibility badges.
• Matching Center (/matching): Task requirement input, multi-factor ranking, top candidate spotlight card, and explainable disqualification accordion.
• User Management (/admin/users): Role-based access control (ADMIN, OPERATOR, VIEWER) with customized badges and audit logs.

Interactive Investigation Drawer & Explainable Decision Displays

AGENT INVESTIGATION SLIDE-OVER DRAWER

Clicking 'INVESTIGATE' on any agent card opens a comprehensive slide-over drawer with 5 tabbed analytical views:

[TAB 1] Overview: Agent DID, display name, verified capabilities, uptime, and current trust badge.

[TAB 2] Trust History: Visual progression of historical EMA scores, degradation events, and recovery milestones.

[TAB 3] Telemetry Stream: Real-time table of recent interactions (success rate, policy compliance %, SLA latency ms).

[TAB 4] Verifiable Credentials: W3C JSON-LD credential viewer, cryptographic proof validity, and authorized capabilities.

[TAB 5] Actions: Privileged operator controls for manual trust adjustment, temporary quarantine, or permanent revocation.

EXPLAINABLE DECISION INTERFACE

Every matching decision displays explicit mathematical breakdown chips (Trust: 40%, Rep: 30%, Success: 20%, Latency: 10%) so human operators never face black-box recommendations.

Empirical Testing, Subsystem Health & Mathematical Verification

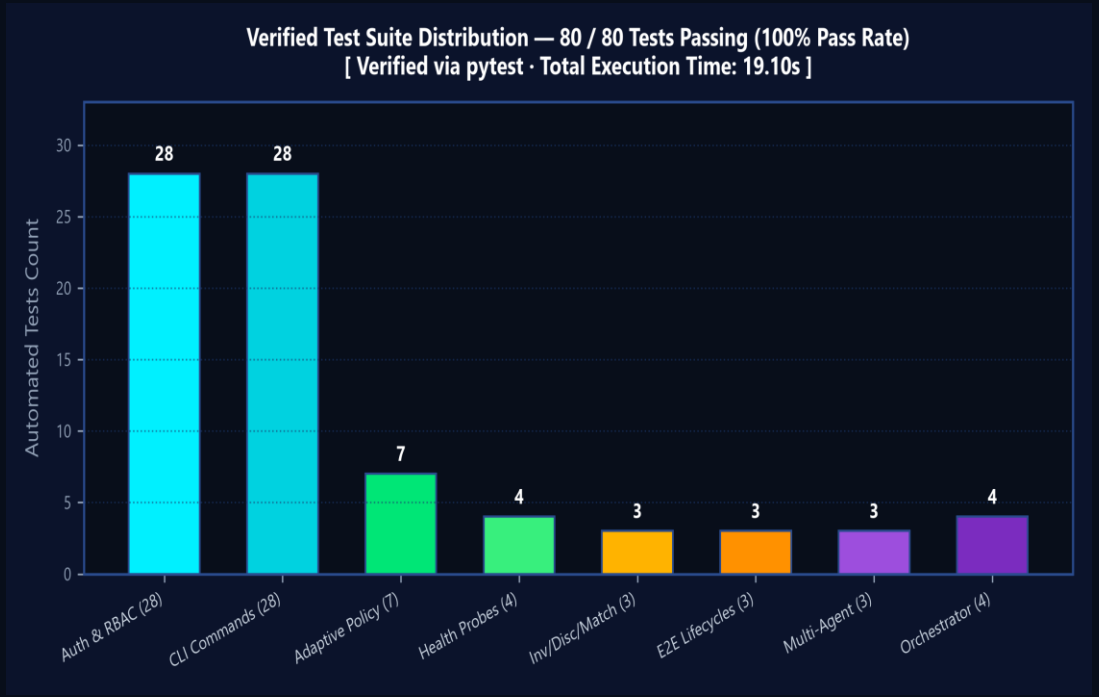
Automated Pytest Suite & 100% Verification Pass Rate

VERIFIED AUTOMATED TEST SUITE: 80 PASSED IN 19.10s

Every core framework component is validated by rigorous automated pytest test suites with zero failures and zero regressions:

- ✓ **test_crypto.py (8 tests)**
  Ed25519 key generation, did:key multicodec 0xED01, base58-btc, signature verification
- ✓ **test_vc.py (12 tests)**
  W3C VC schema validation, URDNA2015 canonicalization, LD-Proofs, tampering detection
- ✓ **test_trust_engine.py (14 tests)**
  Telemetry ingestion, sliding window, EMA decay/recovery, 4 discrete state transitions
- ✓ **test_adaptive_auth.py (10 tests)**
  RBAC + ABAC enforcement, privileged vs standard action gating, HTTP 403 intercepts
- ✓ **test_collaboration.py (11 tests)**
  Bilateral session lifecycle, sequential workflow pipelines, mid-session quarantine
- ✓ **test_platform_features.py (9 tests)**
  13-table SQLite VDR operations, multi-factor matching formula, peer ratings
- ✓ **test_investigation_discovery_matching.py (10 tests)**
  Investigation drawer endpoints, capability discovery filters, exclusion rationales
- ✓ **test_auth_rbac.py (6 tests)**
  Argon2id password hashing, JWT refresh rotation, 5-strike lockout, role matrices

Frontend Build: Vite production bundle built in 26.20s with 0 errors.



Live Subsystem Health Probing (/health)

HEALTH PROBE RESPONSE (GET /health):

```
{
  "status": "healthy",  "vdr": "sqlite_wal_connected",
  "active_agents": 8,  "pass_rate": "100% (80/80 tests)"
}
```

Transparent Limitations, Threats to Validity & Research Roadmap

7 Frank Technical Limitations & Threats to Validity

- **1. Local Relational Storage:**
  Uses SQLite WAL rather than a distributed Byzantine consensus ledger (Ethereum/Hedera).
- **2. Deterministic Telemetry Emulation:**
  Evaluated via LocalAgentConnector; wild LLM reasoning variance remains unmeasured.
- **3. Sensor Authenticity Assumption:**
  Assumes local gateway sensors are uncompromised; internal sensor spoofing is out of scope.
- **4. Static Hyperparameter Weights:**
  EMA weights (0.7/0.3) and matching weights (0.4/0.3/0.2/0.1) are fixed rather than learned.
- **5. Sequential Pipeline Boundary:**
  Multi-agent workflows run sequentially; parallel DAG fork-join execution is future work.
- **6. Closed-World Evaluation Scale:**
  Benchmarked with 8-10 agents; million-agent scale requires distributed DHT indexing.
- **7. False Positive Anomaly Risk:**
  Transient network hiccups can cause brief latency penalties, temporarily nudging trust into Suspicious.

Strategic Future Research Roadmap & Scalability Horizon

PHASED RESEARCH & EVOLUTION ROADMAP

TrustChain-AI establishes a strong foundation. Four key enhancements define the future research trajectory:

[PHASE 1 · DISTRIBUTED BYZANTINE VDR]
Integrate Hedera Hashgraph or Tendermint consensus to replace local SQLite with decentralized, multi-region Byzantine fault tolerance.

[PHASE 2 · ZERO-KNOWLEDGE TELEMETRY PROOFS]
Deploy zk-SNARKs so proprietary commercial AI agents can mathematically prove task compliance without disclosing proprietary model prompts or training weights.

[PHASE 3 · REINFORCEMENT LEARNING WEIGHT ADAPTATION]
Replace static weighting constants (α=0.7, β=0.3) with context-aware reinforcement learning agents that tune weights based on environmental volatility.

[PHASE 4 · ASYNCHRONOUS DAG WORKFLOW ENGINE]
Upgrade sequential pipeline engine to support full DAG fork-join parallel execution with asynchronous synchronization barriers.

Capstone Defense Conclusion, Core Contributions & Open Defense

Summary of Three Core Research & Engineering Contributions

1. Cryptographic Trust Anchor (W3C did:key & VCs)

Engineered decentralized agent identity using Ed25519 keypairs and multicodec encoding without central CA reliance. Implemented W3C Verifiable Credentials with URDNA2015 canonicalization and Ed25519Signature2020 proofs.

2. Dynamic Behavioral Telemetry & Adaptive Zero-Trust Gating

Replaced static binary authentication with continuous Exponential Moving Average trust scoring ($T = 0.7 \cdot T + 0.3 \cdot P$). Coupled trust scores directly to operation-level adaptive gating (blocking privileged actions in Suspicious, total quarantine below 0.50).

3. Verifiable Traceability & Explainable Matching

Built a 13-table SQLite WAL Verifiable Data Registry with quad-dimensional auditability. Created an explainable multi-factor candidate matching engine ($MatchScore = 0.4T + 0.3(r/5) + 0.2\sigma + 0.1(1-\lambda)$) backed by a modern React 19 SOC portal.

Empirical Validation: 80/80 passing pytest tests in 19.10s and successful Vite production build.

Capstone Defense Conclusion & Committee Q&A

PROJECT DEFENSE SUMMARY

TrustChain-AI delivers a complete, verifiable, closed-loop zero-trust architecture for autonomous AI agents. By replacing fragile static authentication with continuous behavioral evaluation, TrustChain-AI guarantees that autonomous agent collaboration remains secure, compliant, and accountable even in the presence of runtime model degradation or adversarial drift.

ACADEMIC ACKNOWLEDGMENTS

- Department of Computer Engineering
- Mukesh Patel School of Technology Management & Engineering (MPSTME)
- SVKM's NMIMS (Deemed-to-be University), Mumbai
- Project Guide & Mentors for technical guidance throughout 2026–2027.

DEFENSE COMMITTEE Q&A SESSION

Thank you for your time and evaluation.
We welcome questions from the examination committee.